



FCT

FACULDADE DE CIÊNCIAS
E TECNOLOGIA
UNIVERSIDADE DOS AÇORES

**FACULDADE DE CIÊNCIAS E TECNOLOGIA DA UNIVERSIDADE DOS AÇORES -
CAMPUS de Ponta Delgada
COORDENAÇÃO DE CURSO DE PÓS-GRADUAÇÃO EM PROGRAMAÇÃO,
ROBÓTICA E INTELIGÊNCIA ARTIFICIAL
Laboratório de Aplicações em Robótica e Aprendizagem**

OpenBalance

Plataforma de Equilíbrio com Visão Computacional e Controlo PID

João Pavão

FCT – UAC

Ponta Delgada, 23 de junho de 2025

João Pavão

OpenBalance

Plataforma de Equilíbrio com Visão Computacional e Controlo PID

Relatório de atividade laboratorial
para avaliação de disciplina
Laboratório de Aplicações em
Robótica e Aprendizagem sob
orientação do Prof. José Cascalho.

FCT – UAC

Ponta Delgada, 23 de junho de 2025

RESUMO

Este relatório apresenta o desenvolvimento do projeto OpenBalance, uma plataforma robótica experimental concebida para equilibrar uma bola em tempo real, controlando dinamicamente a inclinação da sua superfície. O sistema recorre a visão computacional (OpenCV) para detetar a posição da bola e aplica algoritmos de controlo PID para mantê-la centrada. Desenvolvido no contexto da unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA), o projeto integra conhecimentos de mecânica, eletrónica, automação e inteligência artificial aplicada. A estrutura física é composta por componentes impressos em 3D, articulações mecânicas e suportes que garantem graus de liberdade bem definidos. No domínio da eletrónica, são utilizados servomotores de alto torque, um driver PCA9685, um Arduino UNO e uma fonte de alimentação estabilizada. A deteção visual e o controlo são integrados numa interface gráfica interativa em Python, desenvolvida com CustomTkinter. Os testes realizados demonstraram estabilidade, resposta eficiente ao controlo e elevado potencial educativo, validando a proposta como recurso didático modular e reproduzível.

Palavras-chave: Visão Computacional, Controlo PID, Arduino, Mecatrónica, Interface Gráfica, Plataforma de Equilíbrio, Python

ABSTRACT

This report presents the development of OpenBalance, an experimental robotic platform designed to balance a ball in real time by dynamically controlling the inclination of its surface. The system uses computer vision (OpenCV) to detect the ball's position and applies PID control algorithms to recenter it continuously. Developed within the scope of the Laboratory of Robotics and Learning Applications (PRIA) course, the project integrates knowledge from mechanics, electronics, automation, and applied artificial intelligence. The physical structure consists of 3D-printed components, mechanical joints, and supports that define specific degrees of freedom. On the electronics side, the system uses high-torque servomotors, a PCA9685 servo driver, an Arduino UNO, and a regulated power supply. Visual detection and control are managed through an interactive graphical interface in Python, built with CustomTkinter. The conducted tests demonstrated system stability, effective response to control inputs, and strong educational potential, validating the platform as a modular and reproducible teaching tool.

Keywords: Computer Vision, PID Control, Arduino, Mechatronics, Graphical Interface, Ball-Balancing Platform, Python

1 INTRODUÇÃO

O projeto **OpenBalance** foi desenvolvido no contexto da unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA), com o objetivo de demonstrar, de forma prática e interativa, os princípios do controlo automático aplicados ao equilíbrio de uma bola sobre uma superfície inclinável.

A plataforma física, com dois graus de liberdade (eixos X e Y), utiliza servomotores de alto torque controlados por um Arduino UNO e um driver PCA9685. A posição da bola é detetada em tempo real por uma webcam USB, com processamento de imagem em Python (OpenCV). Os ângulos de inclinação são definidos por um controlador PID, com parâmetros ajustáveis por uma interface gráfica desenvolvida em CustomTkinter. Concebido com foco pedagógico e modularidade, o **OpenBalance** visa facilitar o ensino de robótica, visão computacional e controlo, permitindo ainda perspetivar evoluções futuras com inteligência artificial aplicada.

2 OBJETIVO

O objetivo principal do projeto é desenvolver uma plataforma robótica educativa que permita explorar conceitos de controlo PID e visão computacional em tempo real, num sistema físico acessível e reproduzível.

Objetivos específicos:

- Construir uma plataforma mecânica com dois eixos de inclinação (X/Y);
- Detetar a posição da bola por webcam e OpenCV;
- Aplicar controlo PID para manter a bola centrada;
- Desenvolver uma interface gráfica com ajuste HSV e controlo PID;
- Calibrar o sistema e otimizar o desempenho;
- Documentar o projeto e disponibilizá-lo em repositório open source;
- Explorar expansões futuras: controlo com mais atuadores, plataforma tipo Stewart, RL e sensores táteis.

3 CONTEXTO

Inserido no curso de pós-graduação em Programação, Robótica e Inteligência Artificial (PRIA), o **OpenBalance** é uma plataforma didática que conjuga hardware acessível e software livre para demonstrar, em tempo real, o funcionamento de sistemas de controlo. O sistema integra uma base física inclinável (PLA+contraplacado), dois servomotores DM996, um Arduino UNO com driver PCA9685 e uma webcam. A

deteção da bola é feita por segmentação de cor via **OpenCV**, e o controlo é realizado por algoritmos **PID** com comunicação serial entre Python e Arduino. A interface gráfica permite controlar o sistema e ajustar parâmetros de forma intuitiva. Este projeto promove a aprendizagem prática de conceitos de robótica e automação, sendo facilmente replicável em ambientes educativos com recursos limitados.

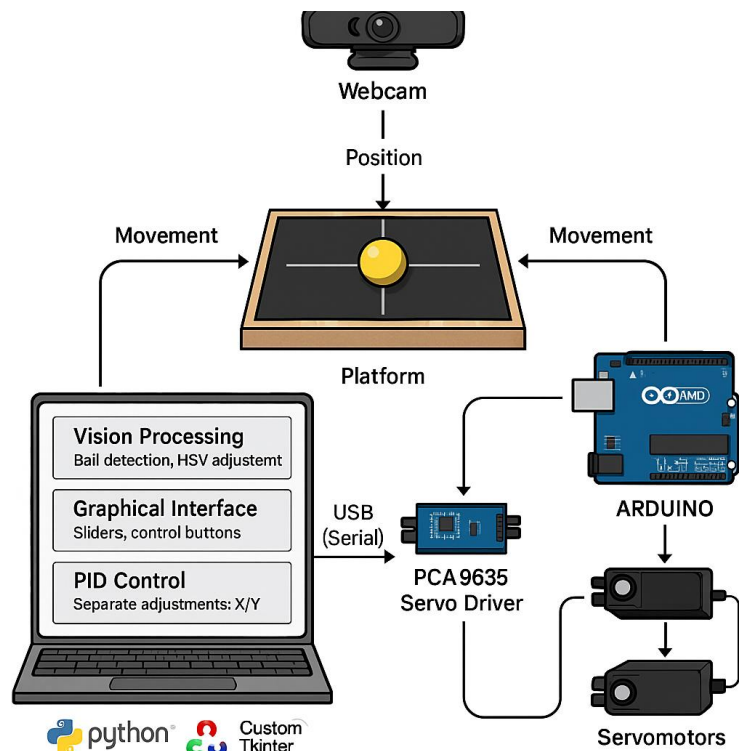


Figura 1 – Diagrama de arquitetura do sistema OpenBalance

4 DESENVOLVIMENTO E RESULTADOS

O projeto **OpenBalance** foi desenvolvido em várias fases, desde a montagem física até à implementação dos sistemas de controlo e interface. A plataforma seguiu princípios de modularidade, acessibilidade e reprodutibilidade, com recurso a componentes simples e ferramentas open source.

Estrutura física e montagem

A estrutura foi construída com base móvel em contraplacado leve e suportes articulados em PLA. Dois servomotores DM996 (15 kg·cm) montados lateralmente permitem controlar a inclinação nos eixos X e Y. A plataforma assenta sobre um pivô côncavo, garantindo estabilidade e liberdade de movimento. Encaixes tipo cavilha nos braços asseguram o alinhamento e evitam deslizamento.

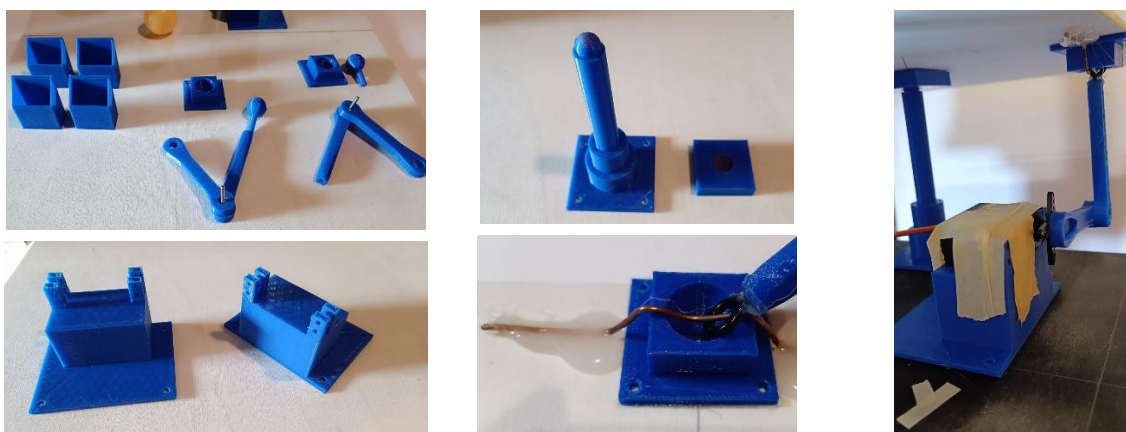


Figura 2 – Vários componentes que formam a estrutura da plataforma

A superfície visível da plataforma foi pintada de preto com uma cruz central (destinada ao alinhamento da webcam e a plataforma) e uma borda de madeira natural (~4 mm), otimizando o contraste visual para a deteção por visão computacional.

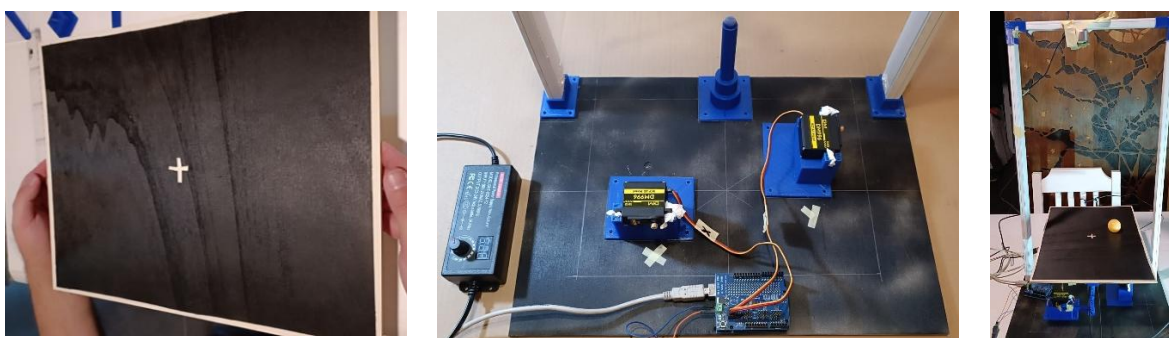


Figura 3 – Montagem da estrutura – etapas finais

Sistema eletrónico

O controlo dos servos é feito por um Arduino UNO ligado a um driver PCA9685 (I²C), que gera sinais PWM precisos. A alimentação é fornecida por uma fonte regulável de 6 V / 72 W, isolando o Arduino da carga. A comunicação com o PC é realizada via porta serial USB. A deteção da bola é feita por uma webcam USB montada verticalmente sobre a plataforma.



Driver PCA9685 e Arduino Uno

Webcam USB

Fonte regulável

Figura 4 – Vários componentes da parte eletrónica

Calibração dos servomotores

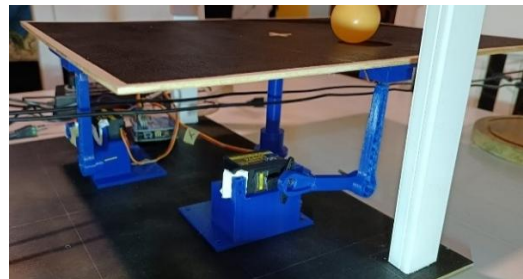
Para garantir movimentos seguros e previsíveis, foi realizada calibração manual dos servos. Cada motor foi centrado a 90°, alinhando a estrutura para obter uma superfície nivelada. Utilizando um sketch de teste, determinaram-se os limites operacionais:

- Eixo X: 80°–120°
- Eixo Y: 70°–110°

Estes valores foram inseridos no **firmware**, assegurando que os comandos respeitem os limites físicos.

```
=== Calibração de Servos via PCA9685 ===  
Comandos: X<ang> ou Y<ang> onde <ang> de 0 a 180  
Exemplos: X0 X90 X180 Y45  
-----  
Servo X -> 95 graus  
Servo X -> 85 graus  
Servo X -> 88 graus  
Servo Y -> 80 graus  
Servo Y -> 85 graus  
Servo X -> 89 graus  
Servo X -> 90 graus  
Servo X -> 92 graus  
Servo Y -> 83 graus
```

Calibração dos motores feita no monitor serial do Arduino IDE



Calibração dos motores de modo a permitir a plataforma nivelada

Figura 4 – Calibração manual dos servomotores

Sintonização PID com o Método de Ziegler–Nichols

A afinação do controlo PID requer definição adequada dos ganhos proporcional (K_p), integral (K_i) e derivativo (K_d). Uma abordagem empírica eficaz, mesmo sem modelo matemático, é o método de Ziegler–Nichols (ZN), amplamente utilizado.

O procedimento consiste em:

1. Desativar K_i e K_d ;
2. Aumentar K_p até provocar oscilações sustentadas ($\rightarrow$ obter K_u);
3. Medir o período de oscilação (T_u);
4. Aplicar as fórmulas empíricas para calcular os ganhos ótimos.

Tabela 1 – Fórmulas empíricas de sintonização segundo Ziegler–Nichols

Tipo de controlador	K_p	K_i	K_d
P	$0.50 \times K_u$	—	—
PI	$0.45 \times K_u$	$1.2 \times K_p / T_u$	—
PID	$0.60 \times K_u$	$2 \times K_p / T_u$	$K_p \times T_u / 8$

Foram realizados testes variando K_p no eixo X, com K_i e K_d a zero e o eixo Y desativado. A análise do comportamento permitiu avaliar a sensibilidade do sistema ao ganho proporcional e orientar a introdução progressiva de K_i e K_d , com base em tentativa e erro.

Tabela 2 – Testes experimentais de controlo proporcional no eixo X

TESTES ao EIXO X										
Teste #	Eixo	Kp	Ki	Kd	Resposta Inicial	Oscilações	Erro Final	Tempo Estabilização	Estável?	Observações
1	X	0.2	0	0	suave	poucas	60 px (-)	pouco		
	X	0.3	0	0	suave	muitas		muito	bola entra em loop	
	X	0.5	0	0	rápida	muitas		muito muito		vai de lado a lado da plataforma
	X	0.4	0	0	media	muitas		muito tempo		toca menos vezes nos lados
	X	0.25	0	0	suave	poucas	113 px (X)	pouco	para ao fim de 10 s	mas toca algumas vezes
	X	0.28	0	0	suave					

Após a afinação individual dos eixos, o controlo PID foi ativado simultaneamente. Os valores atualmente utilizados no firmware estão ilustrados na Figura 5 e servem como referência inicial, podendo ser ajustados conforme a resposta do sistema.

```
// ===== Parâmetros PID por eixo, obtidos pelo método ZN =====
// -----EIXO X-----
const float Kp_X = 0.18, Ki_X = 0.015, Kd_X = 0.07;
// Sugestão: aumenta Kp_X para resposta mais rápida.
// Se houver oscilações, aumenta Kd_X ou reduz Kp_X.

// -----EIXO Y-----
const float Kp_Y = 0.14, Ki_Y = 0.015, Kd_Y = 0.05;
// Sugestão: se o eixo Y parecer mais lento,
// testa aumentar Ki_Y para 0.02 ou subir Kp_Y ligeiramente.
// =====
```

Figura 5 - Parâmetros PID definidos no firmware Arduino com base no método ZN

Está prevista uma futura atualização da dashboard com funcionalidade ZN, que estimará automaticamente os ganhos PID a partir do comportamento oscilatório, simplificando e acelerando o processo de afinação.

Interface gráfica e interatividade

A interface gráfica foi desenvolvida em **CustomTkinter**, com arquitetura modular e tema escuro. É composta por três áreas principais:

- **Painel HSV** – sliders para ajuste em tempo real da segmentação por cor, com presets e visualização da máscara;
- **Área de vídeo** – exibe a imagem processada com a bola detetada, erro X/Y e marcadores de referência;
- **Controlo do sistema** – botões para gerir motores, seguimento automático, redefinir o centro e escolher a porta COM.



Figura 6 – Várias partes constituintes da interface gráfica

A versão 1.0 da dashboard inclui:

- Janela popup com gráficos em tempo real (Erro X/Y vs Tempo e Trajetória Y vs X);
- Resolução de vídeo 640×480, equilibrando qualidade e desempenho;
- Barra de estado com mensagens de sistema e erros;
- Botão "Reset Centro", que redefine visualmente o ponto de convergência da bola com um clique, facilitando os testes.

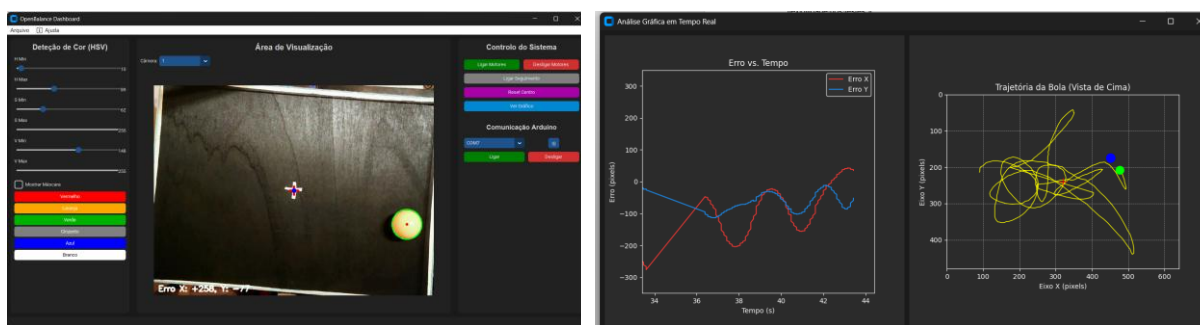


Figura 7 - Interface gráfica da dashboard OpenBalance (v1.0) e Análise gráfica em tempo real

A aplicação permite guardar/carregar configurações HSV (JSON), alternar entre câmaras e aceder a menus de ajuda. Foi concebida como ferramenta robusta, pedagógica e extensível para afinação e demonstrações educativas.

Testes e resultados preliminares

Os testes realizados em ambiente real revelaram:

- **Deteção visual** eficaz com bolas contrastantes (ex: laranja), a 20–25 FPS sob luz difusa;
- **Estabilização** confiável após perturbações, com PID ajustado por Ziegler–Nichols;
- **Resposta física** precisa dos servos, com limites que evitam colisões;
- **Afinação assistida**, facilitada pela visualização em tempo real dos efeitos dos parâmetros.

O projeto está documentado num repositório GitHub com vídeos, presets e imagens:
<https://github.com/joaopavao1979/OpenBalance>

Procedimento futuro para Aplicação de Algoritmos de Otimização e Aprendizagem

Para integrar algoritmos de otimização e aprendizagem no **OpenBalance**, propõe-se um procedimento experimental com aplicação direta no sistema físico, útil para afinar controladores **PID** ou treinar agentes de **Reinforcement Learning (RL)** com base em dados reais.

1. Definir parâmetros e intervalos:

Selecionar os parâmetros a otimizar (ex: K_p , K_i , K_d ou políticas de ação), definindo limites seguros.

2. Preparar o sistema:

Garantir estabilidade e segurança (ex: proteção contra quedas). A comunicação Arduino–Python deve aceitar envio dinâmico de parâmetros, com visão ativa.

3. Estruturar episódios experimentais:

Cada episódio inclui:

- Envio de parâmetros;
- Execução por tempo fixo ou até falha;
- Cálculo de métricas (erro, tempo até falha, número de colisões);
- Armazenamento dos dados.

4. Critérios de término:

Encerrar o episódio ao detetar colisão ou atingir o tempo-limite.

5. Aplicar o algoritmo:

Possíveis abordagens:

- **Hill Climbing** – variações incrementais com seleção por desempenho;
- **Q-Learning** – construção de política discreta com base em recompensas;
- Outros ...

6. Registo e visualização:

Guardar os resultados e gerar gráficos de evolução e comparação de estratégias.

7. Validação final:

Testar a solução considerada ótima em vários episódios para avaliar robustez e consistência. Este procedimento fornece uma base sólida para aplicação de algoritmos de controlo inteligente em robótica, permitindo explorar IA de forma prática.

5 CONCLUSÃO

O projeto **OpenBalance** permitiu aplicar, de forma prática e integrada, os conhecimentos adquiridos na unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA). Através da construção de uma plataforma física capaz de equilibrar uma bola com base em visão computacional e controlo PID, foram explorados conceitos fundamentais de automação, eletrónica e programação, perspetivando-se futuramente a integração de inteligência artificial aplicada.

Privilegiando simplicidade, modularidade e reprodutibilidade, a plataforma utilizou componentes acessíveis e software open source. A interface gráfica desenvolvida em Python com OpenCV e CustomTkinter, aliada à comunicação com o Arduino, revelou-se uma solução eficaz para controlo e afinação em tempo real, com clara mais-valia para contextos didáticos.

Do ponto de vista pedagógico, o sistema demonstrou elevado potencial como recurso de ensino em áreas como controlo automático, visão computacional e robótica, favorecendo a aprendizagem ativa e a ligação entre teoria e prática.

No entanto, o projeto enfrentou desafios importantes. A calibração física da plataforma revelou-se mais complexa do que o esperado, sendo difícil realizar a afinação Ziegler–Nichols por eixo e combinar ambos de forma linear. Também surgiram limitações na estrutura mecânica, como a fragilidade dos braços e uniões da plataforma, que exigiram reforços improvisados com cola quente, fita adesiva e ajustes manuais. A montagem exigiu várias tentativas e erros, especialmente ao alinhar os ângulos neutros dos servos. Apesar destas dificuldades, a experiência foi profundamente formativa, destacando a importância da prototipagem, da integração entre hardware e software e da capacidade de adaptação perante constrangimentos reais. Salienta-se ainda o valor da cultura **maker**, que deve ser mais promovida nos clubes de robótica, incentivando a experimentação prática, a criatividade e a resolução colaborativa de problemas.

Perspetivas Futuras

A experiência adquirida com o **OpenBalance** abre várias possibilidades de evolução:

- Expansão para sistemas com mais graus de liberdade;
- Inclusão de braços articulados capazes de interagir ativamente com a bola, promovendo comportamentos orientados a objetivos;
- Integração de algoritmos de aprendizagem por reforço, permitindo adaptação progressiva baseada na experiência;
- Substituição da visão computacional por superfícies táteis ou resistivas,

melhorando a latência e fiabilidade da deteção;

- Uso de câmaras de maior resolução e processamento embarcado com Raspberry Pi, aumentando a autonomia e portabilidade.

A nível pessoal e académico, o projeto reforçou a importância da experimentação, da integração entre hardware e software e do potencial das ferramentas open source em contextos educativos. No futuro, pretendo continuar a explorar formas de tornar sistemas como o **OpenBalance** mais autónomos, interativos e adaptativos, integrando aprendizagem automática e novas modalidades sensoriais.

BIBLIOGRAFIA

Referências Bibliográficas – Artigos Científicos

- Pradeepa, S. C., Abhijith, N., Amoghavarsha, S. G., Kiran, C. N., & Niranjana Jois, H. C. (2022). *Ball balancing PID system using image processing*. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 10(VIII), 397–406. <https://doi.org/10.22214/ijraset.2022.46190>
- Frank, A. H., & Tjernström, M. (2019). *Construction and theoretical study of a ball balancing platform: Limitations when stabilizing dynamic systems through implementation of automatic control theory* (Bachelor's thesis, KTH Royal Institute of Technology). TRITA ITM-EX 2019:35. <https://www.kth.se>
- Maldonado, A., Bueno, C., Loza, D., & Ibarra, A. (s.d.). *Design, construction and implementation of Stewart Platform – Control of rolling ball on platform through artificial vision*. Universidad de las Fuerzas Armadas – ESPE.
- Ziegler, J. G., & Nichols, N. B. (1942). *Optimum settings for automatic controllers*. *Transactions of the ASME*, 64, 759–768. Disponível em <http://www.driedger.ca>

Controlo e Automação

- Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.
- Dorf, R. C., & Bishop, R. H. (2017). *Modern Control Systems* (13th ed.). Pearson.
- Nise, N. S. (2020). *Control Systems Engineering* (8th ed.). Wiley.

Robótica Educacional e Arduino

- Blum, J. (2013). *Exploring Arduino: Tools and Techniques for Engineering Wizardry*. Wiley.
- Monk, S. (2016). *Programming Arduino: Getting Started with Sketches* (2nd ed.). McGraw-Hill Education.

Python, OpenCV e Interfaces Gráficas

- Beyeler, M. (2017). *Machine Learning for OpenCV*. Packt Publishing.
- Laganière, R. (2014). *OpenCV 3 Computer Vision Application Programming Cookbook*. Packt Publishing.
- Chen, J. (2023). *Learn OpenCV with Python by Examples*.
- Harrison, M. (2017). *Tkinter GUI Programming by Example*. Packt Publishing.

Webgrafia

- GitHub – JohanLink:
<https://github.com/JohanLink/Ball-Balancing-PID-System>
- GitHub – giusenso:
<https://github.com/giusenso/Ball-Balancing-PID-System>
- Thingiverse – Ball Balancing PID Robot:
<https://www.thingiverse.com/thing:4457405>
- Documentação oficial do OpenCV:
<https://docs.opencv.org>
- Adafruit – 16-Channel PWM Servo Driver (PCA9685):
<https://learn.adafruit.com/16-channel-pwm-servo-driver>
- CustomTkinter – GUI Moderna para Python:
<https://github.com/TomSchimansky/CustomTkinter>
- Control Engineering – PID Control Explained:
<https://www.controleng.com/articles/pid-control-explained>
- Maker Guides – Curso de Controlo PID com Arduino:
<https://www.makerguides.com/arduino-pid-controller>

Utilização de Inteligência Artificial como Ferramenta de Apoio

Durante a elaboração deste projeto, foram utilizadas ferramentas de Inteligência Artificial generativa como **ChatGPT** (OpenAI), **Gemini AI** (Google AI Studio) e **Grok** (xAI), com o objetivo de apoiar a estruturação do relatório, esclarecer dúvidas técnicas e sugerir soluções programáticas. Estas ferramentas serviram como complemento ao trabalho do autor, que foi responsável pela interpretação, implementação e validação prática de todas as soluções propostas.

Exemplos de prompts utilizados:

- “Cria uma estrutura de relatório técnico para um projeto de controlo PID com visão computacional.”
- “Explica como calibrar servos com Arduino usando PCA9685.”
- “Como implementar controlo PID em Python com comunicação serial para Arduino?”
- “Sugere uma interface gráfica com CustomTkinter para controlo PID e HSV em tempo real.”
- “Descreve os passos de testes e validação de um sistema que equilibra uma bola com feedback visual.”
- “Adiciona perspetivas futuras para uma plataforma de controlo com múltiplos braços e IA.”
- “Faz uma reflexão final para relatório técnico com foco pedagógico e open-source.”

Estas interações fazem parte do processo de **engenharia de prompts**, uma prática recente e relevante na utilização eficaz de sistemas de IA, que foi conscientemente explorada como ferramenta de aprendizagem e prototipagem rápida.

Ferramentas de IA utilizadas:

- **ChatGPT** – OpenAI:<https://chat.openai.com>
- **Gemini AI Studio** – Google:<https://studio.google.com>
- **Grok** – xAI:<https://x.com>